

Distributed GIS Project

Requirements Compliance Report

Weather-Aware Road Risk Mapping Platform for Berlin
Comparison with *Distributed GIS 00 - Organization* project requirements

Overall assessment	The current project satisfies the mandatory structural components and the core mandatory use cases. It also implements several optional elements.
Architecture	Leaflet/JavaScript web client -> FastAPI REST API -> PostgreSQL/PostGIS, orchestrated with Docker Compose.
Analysis	Client-side JavaScript risk/distance analysis and server-side PostGIS spatial analysis are both present.
Not implemented	Authentication/login and raster/GeoTIFF processing are not implemented; both are optional in the course slides.

1. Course Requirements Used for This Comparison

The course requires a web client, a spatial database backend, a RESTful API middleware, and Docker-based deployment including the database, API and web client. The mandatory use cases additionally require map exploration, persistent user-created spatial data that can be changed later, and analysis on both the client and server side.

Optional components include mobile-friendly design, Git-based version control, custom or external basemaps, raster integration, external services/live feeds, and authentication/user management.

2. Mandatory Structural Components

Course requirement	Implementation in our project	Status
Web client	HTML/CSS/JavaScript application using Leaflet. The user can navigate the map, display spatial layers, interact with reports and edit report data.	MET
Mapping component	Leaflet is used for interactive mapping. OSM and Esri Satellite are available as basemaps.	MET
Display/query/edit spatial data	Hazard reports, risk roads, heatmap/weather layers and report markers are displayed. Reports can be created, inspected, edited and deleted.	MET
Spatial database backend	PostgreSQL with PostGIS stores road reports and the imported Berlin road network using spatial geometries.	MET
Appropriate database schema	Spatial tables include road_reports and road_network. Reports store geometry plus hazard, severity, road/weather and descriptive attributes.	MET
RESTful API middleware	FastAPI provides REST endpoints between the Leaflet client and PostgreSQL/PostGIS, including report CRUD, weather, location search and road analysis.	MET
Retrieve/create/update spatial data	GET /reports retrieves reports; POST /reports creates them; PUT /reports/{id} updates them. DELETE is additionally implemented.	MET
Docker deployment	Docker Compose provides three core services: PostgreSQL/PostGIS database, FastAPI backend and Nginx web frontend. An auxiliary road-importer loads Berlin OSM data.	MET
Easy execution	START_PROJECT.bat automates Docker checking, Berlin OSM data availability, database startup, road import and application startup on Windows.	MET

3. Mandatory Use Cases

Mandatory use case	Evidence / project example	Status
Explore a map with basemap and topic-related features	Leaflet provides interactive navigation with OSM/Esri basemaps and topic layers including user reports, risk roads, heatmap and weather information. Risk legend and layer controls support interpretation.	MET
Create and permanently store spatial data	A user can add a road hazard report. The report is sent through FastAPI and stored persistently in PostgreSQL/PostGIS.	MET
See the data again and change it later	Stored reports are loaded from the server on later visits and can be edited or deleted.	MET*
Client-side analysis	JavaScript functions such as calculateDistanceMeters(), getWeatherFactor(), getPointRoadAnalysis() and buildRiskRoadSegments() combine location, severity, distance and weather factors to derive risk information in the browser.	MET
Server-side analysis	FastAPI/PostGIS uses ST_DWithin, ST_Distance, ST_ClosestPoint, ST_Buffer, ST_Intersection and ST_Length and calculates weather-adjusted road risk scores.	MET

***Note on 'own contributions':** Persistent creation, retrieval, editing and deletion are implemented, but reports are not separated by authenticated user account. Authentication is separately described by the course as optional/encouraged, so this is documented transparently rather than treated as a missing mandatory structural component.

4. Concrete Analysis Examples

Client-side example

In the browser, JavaScript calculates distances between road positions and reports, applies severity-dependent influence distances and weather factors, and derives risk scores. Road geometry can be segmented and visualized according to the calculated risk. The calculation is performed directly by JavaScript in the web client.

Server-side example

On the server, PostGIS determines spatial relationships between road geometries and reports. ST_DWithin identifies nearby roads, ST_Buffer creates influence areas, ST_Intersection extracts affected road sections, and ST_Length measures them. Severity and weather conditions are also used in the server-side risk score.

5. Optional Structural Components

Optional component	Project status	Assessment
Git / version control	GitHub is used; the final working development version is maintained in the berlin-final branch.	IMPLEMENTED
External basemaps	OpenStreetMap and Esri Satellite basemaps are integrated.	IMPLEMENTED
External services / live data	Weather data and geocoding/location services are integrated; OSM is used as a geospatial source.	IMPLEMENTED
Authentication / user management	No OAuth/JWT/login system is currently implemented.	OPTIONAL - NOT IMPLEMENTED
Raster / GeoTIFF processing	No dedicated raster-analysis workflow is implemented.	OPTIONAL - NOT IMPLEMENTED
Custom map server	No dedicated GeoServer/MapServer is needed because external basemaps are used.	NOT REQUIRED

6. Architecture Mapping

Web client: Nginx serves the HTML/CSS/JavaScript Leaflet application. It visualizes spatial information, receives user input, calls REST endpoints and performs browser-side analysis.

Middleware/API: FastAPI exposes the REST interface and connects application logic to the spatial database.

Spatial backend: PostgreSQL/PostGIS stores persistent report and road data and performs server-side spatial operations.

Containerization: Docker Compose connects the web client, API and database services on a shared Docker network.

WEB CLIENT
Leaflet + JavaScript
Map interaction + client-side analysis

REST / GeoJSON

FASTAPI
REST middleware / application API

SQL / spatial queries

POSTGRESQL + POSTGIS
Persistent spatial data + server-side analysis

7. Final Assessment

No major mandatory project component is currently missing. The application demonstrates the distributed architecture required by the course: an interactive JavaScript/Leaflet web client, REST middleware, a PostGIS spatial database and Docker-based deployment. It also demonstrates both client-side and server-side analysis.

The only qualification is that reports are not associated with authenticated individual user accounts. The application nevertheless demonstrates persistent creation and later modification of user-generated spatial objects. Since authentication/user management is presented as optional/encouraged in the organization slides, this should be explained transparently rather than treated as a missing mandatory structural component.

Recommended demonstration: create a hazard report -> store it in PostGIS -> show road-risk analysis -> explain which calculation happens in JavaScript and which spatial operations happen in PostGIS -> reload/edit the stored report -> briefly show the three Docker services.

Basis of assessment: Distributed GIS 00 - Organization project requirement slides and the current berlin-final implementation.